TP1 - Git avanzado

Materia: Metodología de Sistemas II
Unidad 4: Repositorios, colaboración y entrega continua
Alumno: Nicolás Viruel

Parte 1: La "Física" de Git

1. Modelo de objetos (blobs, trees, commits)

• Blob: guarda el contenido binario de un archivo (sin nombre ni permisos). Varios paths pueden apuntar al mismo blob si el contenido es idéntico.
• Tree: es un directorio versionado: lista entradas (nombre, modo, puntero a blob u otro tree). Representa la estructura de carpetas de un snapshot.
• Commit: es un nodo del historial: apunta a un tree (estado del proyecto), incluye autor, fecha, mensaje y padre(s). Forma el DAG.

Juntos: el commit → tree raíz → trees/blobs = instantánea reproducible del repositorio en ese punto.

2. Direccionamiento por contenido

La “dirección” de un objeto es su hash SHA-1/SHA-256 calculado sobre su contenido y metadatos. Si cambia un byte, cambia el hash → integridad: cualquier alteración se detecta al resolver referencias. La deduplicación ocurre porque dos archivos iguales generan el mismo blob y se almacenan una sola vez en .git/objects.

3. Grafo dirigido acíclico (DAG)

Cada commit referencia el hash de su padre. Si modifico un blob del pasado, cambian el tree y todos los commits descendientes (nuevos hashes). Eso rompe la cadena hacia adelante: es una cadena de custodia criptográfica - el historial publicado deja de coincidir con el recalculado, señal de manipulación o reescritura.

Parte 2: Áreas de trabajo y flujo atómico

4. Secuencia git add → git commit

1. Working directory: archivo modificado.
2. git add: calcula el blob, lo escribe en .git/objects si no existe y actualiza el index (staging area) con la entrada (path, modo, hash del blob).
3. git commit: construye un tree desde el index, crea el commit que apunta a ese tree y al commit padre, y mueve la rama (p. ej. main) al nuevo hash.

El staging permite seleccionar qué entra en el próximo commit sin commitear todo el working tree.

5. HEAD y detached HEAD

HEAD es la referencia al commit “actual” (normalmente vía nombre de rama). En detached HEAD apunta a un commit concreto, no a una rama: los nuevos commits no avanzan ninguna rama y pueden quedar huérfanos al cambiar de checkout. Utilidad: inspeccionar tags, probar un commit antiguo o hacer un hotfix experimental; para resiliencia, conviene crear rama temporal si se va a commitear.

Parte 3: Gestión y cirugía del historial

6. Limpieza de historial

• Para unir 5 commits locales: git rebase -i HEAD~5 (o HEAD~n) y marcar commits como squash / fixup, dejando un mensaje profesional final.
• git commit --amend: reescribe solo el último commit (mensaje o contenido del index) sin cambiar la base del historial previo.
• Rebase interactivo: reordena, fusiona o edita varios commits; reescribe hashes desde el punto de rebase (no usar en commits ya compartidos sin acuerdo).

7. Cherry-pick vs merge

Cherry-pick aplica un commit concreto de otra rama sobre la rama actual (nuevo hash, mismo diff). Preferible en hotfix: en main está la versión estable; en feature existe el commit abc123 que corrige la falla de producción. Con git cherry-pick abc123 traigo solo ese arreglo sin mergear toda la feature incompleta.

Parte 4: Resiliencia

8. Escenario git reset --hard HEAD~1

A) ¿Está perdido? No necesariamente. reset --hard mueve la rama y el working tree, pero el commit anterior sigue en el reflog (referencias locales recientes) hasta que expire el garbage collection.

B) Recuperación con reflog:

1. git reflog → identificar el hash del commit antes del reset (p. ej. a1b2c3d).
2. git checkout -b recuperacion a1b2c3d o git reset --hard a1b2c3d si aún no se hizo push destructivo en remoto.
3. Verificar con git log y tests; luego push o merge según política del equipo.

9. Reset vs revert (commit ya pusheado)

Reset reescribe historial compartido → obliga a force push y rompe clones de compañeros. Alternativa segura: git revert <commit>, que crea un nuevo commit con el inverso del cambio, preservando el historial público y permitiendo trazabilidad en auditorías.

Parte 5: Diagnóstico (plumbing)

10. Comandos de bajo nivel

• Verificar tipo de objeto: git cat-file -t <hash> (devuelve blob, tree, commit, tag).
• Ver contenido: git cat-file -p <hash> (pretty-print del objeto; útil para inspeccionar tree/blob sin checkout).
• Complemento: git fsck para detectar objetos corruptos o dangling.